

LLM AGENTS — LANGCHAIN & LANGGRAPH

O'Reilly Live Training — Complete Reference Cheatsheet

Notebooks 0.0 – 7.0 | LangGraph 1.0+ | Python 3.11+

0. WHY AGENTS? — THE AGENT LOOP FROM SCRATCH

What is an Agent?

An **agent** = LLM + Tools. The model decides when and how to call tools, gaining access to the real world (search, files, APIs, code execution).

- **LLM:** reasoning, planning, language
- **Tools:** real-world actions & data
- **Loop:** keep going until goal is met

OpenAI Function Calling

- Define tools as **JSON schemas** (name, description, parameters)
- Pass schemas to model; model returns **tool call objects**
- Your code executes the tool, feeds result back
- Key: model decides *whether* and *when* to call tools

The Manual Agent Loop



Key insight: AgentExecutor IS a graph under the hood — LangGraph makes this loop explicit and controllable.

1. INTRO TO LANGGRAPH — WHY A FRAMEWORK?

Why LangGraph over Manual Loops?

- **State management** — built-in, typed
- **Cycles** — retry and loop natively
- **Conditional routing** — branch on any condition
- **Streaming** — token-by-token and step-by-step
- **Persistence** — checkpoints between steps
- **Observability** — LangSmith tracing built-in

LangChain Building Blocks

- **Model access:**
`ChatOpenAI(model="gpt-4o")`
- **Prompt templates:** reusable strings with dynamic variables
- **Chains:** composable LLM + prompt blocks
- **Structured outputs:**
`model.with_structured_output(MyModel)`
- **Model + tools:** `model.bind_tools(tools)`
- **Pydantic:** validate structured output schemas

2. LANGGRAPH CORE CONCEPTS — STATE · NODES · EDGES

State

Shared data structure (TypedDict or Pydantic model) — the current snapshot of your application. Passed into every node; returned updates are merged back.

Nodes

Python functions that receive the current State as input, perform computation or side-effects, and return a dict of updated State fields.

Edges

Connections between nodes. Fixed edges always go to the same next node. Conditional edges call a routing function to choose dynamically.

Building a Basic Graph

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict

class MyState(TypedDict):
    messages: list
    count: int

def my_node(state: MyState) -> dict:
    return {"count": state["count"] + 1}

def router(state: MyState) -> str:
    return "done" if state["count"] >= 3
    else "again"

graph = StateGraph(MyState)
graph.add_node("worker", my_node)
graph.add_edge(START, "worker")
graph.add_conditional_edges(
    "worker", router, {"again":
"worker", "done": END}
)
app = graph.compile()
```

MessagesState & Reducers

Use `MessagesState` for conversation history — it automatically **appends** messages instead of replacing the list.

```
from langgraph.graph import MessagesState

class AgentState(MessagesState):
    pass # add extra fields here
```

Custom reducers: annotate fields with `Annotated[list, add_messages]` or your own merge function.

Command Primitive (Post-1.0)

Command combines state update + routing in a single return. Replaces the dict-return + separate conditional edge pattern.

```
from langgraph.types import Command

def my_node(state):
    return Command(
        update={"result": "done"},
        goto="next_node" # or END
    )
```

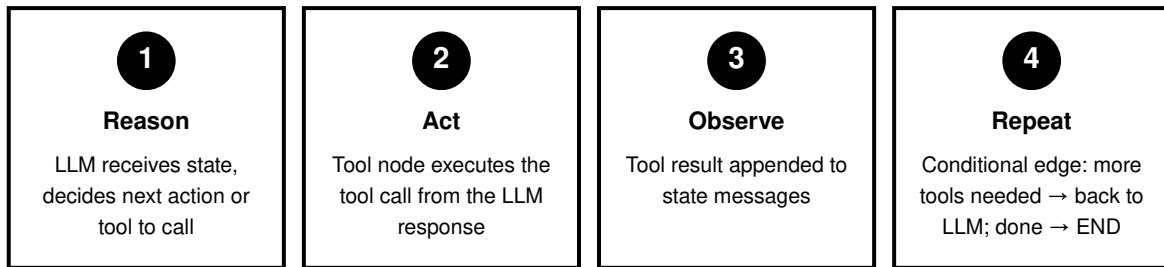
Multi-Schema State

A graph can have separate schemas for three purposes:

- **Internal state:** all fields needed during execution
- **Input schema:** what callers provide (`input_schema=`)
- **Output schema:** what the graph returns (`output_schema=`)

Ask: what do I need to *track* vs. what do I need to *expose*?

REACT AGENT LOOP



3. LOCAL RESEARCH AGENT (LANGGRAPH + OLLAMA)

Research Agent Pattern

- 1 Generate search queries from user question
- 2 Perform web searches (Tavily, SerpAPI)
- 3 Summarize search results
- 4 Reflect — identify knowledge gaps
- 5 Repeat if gaps remain, else return answer

Key Patterns

- **ConfigurableRunnableConfig**: parameterize agents (model choice, depth) without re-compiling the graph
- **Multi-tool prompts**: explicit tool descriptions in system prompt improve reliability
- **RAG chunking**: recall > precision — a missed chunk can't help; semantic chunking beats fixed-size for retrieval quality
- **Local LLMs**: use Ollama for offline / private data

4. LANGGRAPH POST-1.0 FEATURES

Functional API (@entrypoint / @task)

Write graphs as regular Python functions using decorators — no explicit node/edge wiring. LangGraph 0.3+

```
from langgraph.func import entrypoint, task

@task
def call_model(messages):
    return llm.invoke(messages)

@entrypoint()
def my_graph(state):
    result =
    call_model(state["messages"]).result()
    return {"messages": result}
```

MCP Integration

Model Context Protocol — connect agents to any MCP server. Install: `pip install langchain-mcp-adapters`

```
from langchain_mcp_adapters.client import (
    MultiServerMCPClient
)
client = MultiServerMCPClient({
    "filesystem": {
        "command": "npx",
        "args": ["-y",
            "@modelcontextprotocol/server-filesystem",
            "/path/to/dir"]
    }
})
tools = await client.get_tools()
```

Durable Execution

- LangGraph Platform provides **resume-on-failure**: if a run is interrupted (network error, timeout, restart), it resumes from the last checkpoint
- Built on the same checkpointer system (see Notebook 5.0)
- No extra code required — configure in `langgraph.json`
- Critical for long-running agents in production

Subgraphs (overview)

- Compile a StateGraph → use it as a **node** in another graph
- Foundation for multi-agent architectures
- Each agent/team is self-contained; parent orchestrates
- Full coverage in Notebook 6.0

5. PERSISTENCE & CHECKPOINTERS

Checkpointer Options

Checkpointer	Use Case	Install	Notes
MemorySaver	Dev / testing	Built-in	Lost on process exit. Never use in prod.
SqliteSaver	Local dev, single-node	<code>langgraph-checkpoint-sqlite</code>	Survives restarts. Good for prototypes.
PostgresSaver	Production	<code>langgraph-checkpoint-postgres</code>	Concurrency, durability, horizontal scale.

Checkpointer Usage Pattern

```
from langgraph.checkpoint.memory import MemorySaver
from langgraph.checkpoint.sqlite import SqliteSaver

# Compile with checkpoint
checkpointer = MemorySaver()
app = graph.compile(checkpointer=checkpointer)

# ALWAYS pass thread_id in config
config = {"configurable": {"thread_id": "user-123"}}

# Same thread_id = same conversation
app.invoke({"messages": [...]}, config=config)

# Inspect state
state = app.get_state(config)
history = list(app.get_state_history(config))

# Override state manually
app.update_state(config, {"messages": [correction]})
```

Cross-Thread Store

Checkpointers are **scoped to a single thread**. For data shared across conversations use the `Store` abstraction:

- User profiles, learned preferences
- Shared knowledge across sessions
- `InMemoryStore` (dev) or external DB (prod)

```
from langgraph.store.memory import InMemoryStore
store = InMemoryStore()
app = graph.compile(
    checkpointer=checkpointer,
    store=store
)
```

6. SUBGRAPHS — MULTI-AGENT COMPOSITION

Pattern 1 — Shared State Keys

When the subgraph's schema **shares keys** with the parent, add the compiled subgraph directly as a node. LangGraph auto-passes matching keys in and merges output keys back.

```
sub = subgraph.compile()
# Add directly — shared keys handled auto
parent.add_node("agent", sub)
```

Simplest and most common pattern.

Pattern 2 — Different Schemas

When schemas differ, wrap the subgraph in a node function that handles translation both ways.

```
sub = subgraph.compile()

def agent_node(state: ParentState):
    # 1. Translate parent → subgraph
    sub_input = {"query": state["input"]}
    # 2. Invoke subgraph
    result = sub.invoke(sub_input)
    # 3. Translate output → parent
    return {"output": result["answer"]}

parent.add_node("agent", agent_node)
```

Persistence in Subgraphs

- Compile the **parent** graph with a checkpointer
- All subgraph invocations within that thread are automatically checkpointed — no extra wiring
- Per-invocation persistence (subgraph has its own checkpointer) is useful for isolated agents

When to Use Subgraphs

- **Multi-agent teams** — each agent is a subgraph; parent orchestrates
- **Reusable modules** — package once, embed in many parents
- **Failure isolation** — a crash in a subgraph doesn't kill the parent
- **Different state schemas** — specialized agents with their own state

7. DEPLOYMENT — LANGGRAPH PLATFORM

Project Structure

```
my-agent/
├── src/
│   ├── my_agent/
│   │   ├── __init__.py
│   │   └── graph.py      # StateGraph
├── def
├── langgraph.json        # Manifest
├── pyproject.toml
└── .env
```

langgraph.json:

```
{
  "dependencies": [".",],
  "graphs": {
    "agent": "./src/my_agent/
graph.py:graph"
  },
  "env": ".env"
}
```

CLI Commands & Deployment Options

Local Development

```
langgraph dev # port 2024, hot-reload, Studio UI
langgraph build # build container image
langgraph deploy # push to LangSmith Deployments
```

Deployment Options

- **Cloud SaaS** — hosted by LangChain (paid)
- **Self-Hosted Cloud (BYOC)** — your infra, their runtime
- **Self-Hosted Enterprise** — full on-prem
- **Local** — `langgraph dev` for dev only

Platform provides: managed runtime, persistence/ threads/assistants APIs, Studio visual debugger, durable execution

KEY APIS — QUICK REFERENCE

Graph Construction

```
from langgraph.graph import (
    StateGraph, MessagesState, START,
    END
)
from langgraph.checkpoint.memory import
    MemorySaver
from langgraph.types import Command

# Build
graph = StateGraph(MyState)
graph.add_node("node_a", fn_a)
graph.add_node("node_b", fn_b)
graph.add_edge(START, "node_a")
graph.add_edge("node_a", "node_b")
graph.add_conditional_edges(
    "node_b", router,
    {"yes": "node_a", "no": END}
)
app = graph.compile(
    checkpoint=MemorySaver()
)
```

Invocation & Inspection

```
# Config with thread_id (required for
checkpointing)
config = {"configurable": {"thread_id":
    "t1"}}

# Invoke (blocking)
result = app.invoke({"messages": [...]},
    config)

# Stream steps
for chunk in app.stream(input, config):
    print(chunk)

# Stream tokens
for token in app.astream_events(input,
    config):
    print(token)

# Inspect / edit state
state = app.get_state(config)
history =
    list(app.get_state_history(config))
app.update_state(config, {"key":
    "value"})

# Command return (node function)
return Command(update={"x": 1},
    goto="next")
```

Prebuilt ReAct Agent

```
from langgraph.prebuilt import
    create_react_agent
from langchain_openai import ChatOpenAI
from
    langchain_community.tools.tavily_search
import (
    TavilySearchResults
)

model = ChatOpenAI(model="gpt-4o")
tools =
    [TavilySearchResults(max_results=3)]

agent = create_react_agent(model, tools)
result = agent.invoke({
    "messages": [("human", "What is
    LangGraph?")]
})
```

Model + Tools Setup

```
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
from pydantic import BaseModel

@tool
def my_tool(query: str) -> str:
    """Search for information."""
    return search(query)

# Bind tools to model
model = ChatOpenAI(model="gpt-4o")
model_with_tools =
    model.bind_tools([my_tool])

# Structured output
class MyOutput(BaseModel):
    answer: str
    confidence: float

structured =
    model.with_structured_output(MyOutput)
```

COURSE SUMMARY

LEARNING PATH — 8 NOTEBOOKS

- **0.0 — Agents from Scratch:** agent loop, OpenAI function calling, tools as JSON schemas, manual ReAct implementation
- **1.0 — Intro LangGraph:** LangChain components (models, prompts, chains), structured outputs, `bind_tools`, AgentExecutor as a graph
- **2.0 — Core Concepts:** State / Nodes / Edges, conditional routing, Command primitive, MessagesState, multi-schema state, full ReAct loop
- **3.0 — Research Agent:** generate-search-summarize-reflect loop, ConfigurableRunnableConfig, multi-tool prompting, local Ollama models
- **4.0 — Post-1.0 Features:** Command, Functional API (`@entrypoint/@task`), MCP integration, durable execution
- **5.0 — Persistence:** MemorySaver → SqliteSaver → PostgresSaver, `thread_id` pattern, state inspection & update, cross-thread Store
- **6.0 — Subgraphs:** shared-key pattern (direct node), different-schema pattern (wrapper fn), persistence in subgraphs, multi-agent architectures
- **7.0 — Deployment:** `langgraph.json` manifest, `langgraph dev` local Studio, `langgraph deploy` to LangSmith, four deployment tiers